

SARUPOS V1.0

Master Technical Documentation

Version 1.0 | 05 September 2026

Architecture • JWT REST API • Database • Billing • E2E Workflow • SaaS Evolution

Table of Contents

- | | |
|--|--|
| 1. System Overview | 8. Frontend / Backend Integration |
| 2. System Architecture | 9. Complete End-to-End Workflow |
| 3. JWT Authentication & RBAC | 10. Architecture Decision Log |
| 4. REST API Architecture | 11. Testing & Validation – Honest Status |
| 5. Database Architecture & Data Integrity | 12. Future SaaS Evolution |
| 6. Billing & Atomic Checkout | 13. Technical Glossary |
| 7. Table & Dining Session State Management | 14. Final Architecture Summary |

1. System Overview



SaruPOS v1.0 is a full-stack **Restaurant Point-of-Sale system** designed around restaurant operations including authentication, table and dining-session management, customer management, POS ordering, billing, payments and receipt generation.

The project is structured as a **React frontend** connected to a **Python/Flask REST API** and a relational **SQLite database**. The architecture separates HTTP route handling from business/service logic and persistence.

2. System Architecture



The system relies on a strictly layered approach to manage requests securely and efficiently.

```
React Frontend → Flask REST API → Authentication/Validation → Route/  
Blueprint Service Layer → SQLite Database → JSON Response → React UI
```

The **route layer** handles HTTP requests, validation and access checks, while business operations are delegated to **service functions**.

3. JWT Authentication & RBAC



Authentication begins at **POST /login**. Credentials are validated and passed to the authentication service. Successful authentication establishes the authenticated user context used for protected API access.

Authentication answers *"who is the user?"* while **authorization/RBAC** answers *"what is this role allowed to do?"*.

```
Login → Credential Verification → JWT → Bearer Token → Protected API  
→ Token Validation → Role Check → Service Layer
```

4. REST API Architecture



SaruPOS exposes functionality through **REST-style HTTP endpoints** using JSON requests and responses. **POST** is used for create, **GET** for read, **PUT** for update and **DELETE** for delete.

A representative protected request follows:

```
HTTP Request → Blueprint Route → Authentication → Role Check / Input  
Validation → Service Function → Database
```

5. Database Architecture & Data Integrity



SQLite provides relational persistence. Core entities include categories, menu items, customers, employees, restaurant tables, bills, bill items, dining sessions and payments.

Primary keys identify records. **Foreign keys** connect related entities. **UNIQUE** constraints prevent duplicates where required, while **NOT NULL** protects required fields.

6. Billing & Atomic Checkout



Checkout is treated as a coordinated business operation to ensure financial accuracy.

```
Validate Order → Obtain Authoritative Menu Pricing → Calculate Totals/  
Tax → Create Bill & Bill Items → Process Payment → Update State →  
Commit
```

If a critical operation fails, the transaction should **roll back** so partially completed checkout data does not remain inconsistent. Server-side pricing and tax keep business-critical totals under backend control.

7. Table & Dining Session State Management

Typical table lifecycle:

```
Available → Occupied → Active Order → Checkout → Payment → Available
```

A **dining session** represents the active operational context associated with a table and its current activity. Closing the completed session and releasing the table is part of completing the restaurant workflow.

8. Frontend / Backend Integration

React handles presentation, interaction, client-side state and API communication.

Flask handles authentication, authorization, validation, business rules, authoritative calculations and database operations.

Integration pattern: React UI → HTTP/JSON API Request → Flask Auth/RBAC/Validation → Service → Database → JSON Response → React UI Update.

9. Complete End-to-End Workflow

The unified process flow defining the core user journey:

1. Login & **JWT Authentication** → Role Verification
2. POS/Dashboard → Table Selection → Dining Session
3. Customer Selection → Order Creation → Menu Items/Quantity
4. Backend Price Validation → Tax Calculation
5. Checkout → Payment (Bill/Bill Items/Payment Record)
6. Receipt Generation → Dining Session Closure
7. Table Release → Available

10. Architecture Decision Log

Flask was used as the Python web framework for the REST backend and modular route organization. **SQLite** was used as a lightweight relational database for the application context. **JWT** provides token-based API authentication, while **RBAC** provides role-aware access control.

A **service layer** separates route handling from business logic. Server-side pricing protects authoritative calculations, and transactional checkout preserves consistency across related billing and payment operations.

11. Testing & Validation – Honest Status

The documentation records validated behavior without overstating automated results. Key end-to-end restaurant workflows were manually exercised, including authentication, table assignment, customer/order flow, payment, receipt generation and table release.

Earlier testing reached a 23-pass result after fixes, while the later production-clean package had remaining automated-test caveats. This document therefore does not claim that the current automated suite is universally green.

12. Future SaaS Evolution

A future SaaS evolution would introduce **tenant-aware data isolation**, cloud deployment, production-grade database infrastructure, centralized authentication, subscription/billing capabilities, monitoring, backups, scalability and stronger operational controls.

Local/Single Restaurant Stable POS Foundation → Tenant-Aware
Architecture → Cloud Backend → Multi-Restaurant SaaS Platform

These are future directions, not current v1.0 implementation claims.

13. Technical Glossary

- **Authentication** - verifying user identity.
- **Authorization** - determining permitted actions.
- **JWT** - token used to carry authenticated context for API access.
- **RBAC** - role-based access control.
- **REST API** - HTTP-based interface for application resources and operations.
- **Blueprint** - Flask mechanism for modular route organization.
- **Service Layer** - business-logic layer called by routes.
- **Primary Key / Foreign Key** - unique identifier and relationship reference between tables.
- **Transaction / Rollback / Commit** - database operation states for data consistency.
- **Multi-tenancy** - serving multiple organizations while isolating their data.

14. Final Architecture Summary

SaruPOS can be understood as a layered operational system:

React UI → REST API → Auth + RBAC + Validation → Modular Routes →
Service Layer → SQLite Persistence → Transactional Business Ops

Key engineering themes are **separation of concerns**, **backend authority**, **role-based security**, **relational data integrity**, **transaction safety** and **end-to-end workflow consistency**.